02 — Engineering & Operations Lessons

Cập nhật 2026-11-04 (branch *hmm-2nd-idea*). Phạm vi: lập trình, cấu hình môi trường, quản lý Kaggle, tổ chức thử nghiệm, lỗi đã gặp và cách sửa, bài học thành công / thất bại. Không chứa kết quả khoa học (→ *01*) hay kế hoạch (→ *03*). Bảng tài khoản và dataset: *ENVIRONMENT_SUMMARY.md*. Bản đầy đủ các vòng trước (ý tưởng 1, SelfCal, GPU L4): *history/02_ENGINEERING_OPS_NOTES_full_through_2026-11-03.md*.

1. Nguyên tắc tối ưu tài nguyên (theo `HMM_Navigation_Yeu_Cau_Nghien_Cuu_Cap_Nhat.docx`, đã kiểm chứng)

Tài nguyên	Thực tế	Quy tắc
Phiên CPU Kaggle	4 vCPU, ~30 GB RAM, 5 phiên / acc, không tốn hạn mức GPU	**Mọi việc của ý tưởng 2 chạy trên CPU** (đánh giá từ cache ~1 ms/frame; sinh cache `nav_demo --cache-only` cũng CPU)
Phiên GPU	2 / acc (T4 hoặc L4×4), 30 GPU-giờ / tuần / acc	Chỉ khi thật cần (trích descriptor mới, luật ý tưởng 1 SH/CST-JS); ước GPU-giờ trước, báo user nếu thiếu
Lấp phiên	—	Lấp hết phiên của một acc rồi mới sang acc kế (thứ tự `automation/active_accounts.txt`); `central_monitor.py` tự làm
Kẹt hàng đợi	—	Không có output 10 phút (T4) / 8 phút (L4, CPU) ⇒ xoá + đẩy lại, tối đa 5 lần, rồi soát lỗi code, rồi 2×T4 (monitor tự làm)
API Kaggle	Giới hạn theo IP (~80 tiến trình hỏi trạng thái ⇒ bị chặn hàng chục phút)	**Một** `central_monitor.py` cho mọi đợt (khởi động lại với cả thư mục cũ và mới; trạng thái trong `state.json` nên không mất gì)
Đĩa phiên chat	~30 GB, có hạn mức riêng	Kéo đầu ra chọn lọc (`--pull-pattern`); xoá bản cục bộ sau khi publish
Song song cục bộ	4 lõi	`OMP_NUM_THREADS` = lõi / số tiến trình; `xargs -P 4`

Luồng chuẩn của một đợt (đã dùng cho mọi đợt `hsi1`–`hsi20`): (1) smoke test cục bộ đầu-cuối trên 1–4 run thiết kế; (2) đóng gói LPT theo chi phí đo (`automation/hsi_wave.py`; job nặng như Boreas tách theo seed); (3) `fanout.py --groups --build-only` (tự đồng bộ code dataset mọi acc); (4) một monitor đẩy / kéo; (5) gom ngay khi từng kernel xong, kiểm `cache_path` trong JSON.

2. Công cụ (ý tưởng 2) — cách dùng

Việc	Lệnh
đánh giá biến thể trên cache	`analysis/hsi_eval.py <cache_dir|DIR:navcache_[@th■ m■c]> <ds> <desc> <seeds> --variants A,B --frames --tag t` → `results/hsi_<ds>_<desc>_s<seed>_<tag>.json` (+ `hsifr_*.npz` mặt nạ)
nhóm job một đợt	`automation/hsi_wave.py <groups.txt> <n_kernels> <variants> --tag t` (13 dataset-seed-group, nguồn `DIR:navcache_@hmm-navcache-v2` / `@hmm-e37-caches`)
dựng kernel	`automation/fanout.py <work> <prefix> <groups.txt> --groups --build-only --cpu [--nodumps] --ds a/b,c/d --par 4 --nocopy`

Việc	Lệnh
theo dõi	`automation/central_monitor.py <state.json> --gap 15 --run-poll 180 --nocopy [--pull-pattern '^(results\.tar|.*\.log)$'] <kernel_dirs...>`
gom kết quả	`automation/hsi_collect.py <out> <run_dir...> --tagre '_e8[.]'` (**bắt buộc** `--tagre`: kết quả chính, lỗ 10/50 % và tập mới trùng khoá dataset/descriptor/seed)
so sánh biến thể	`analysis/hsi_compare.py <dir> A,B,C [--base <oc dirs>] [--all] [--md out]` (mặc định chỉ run phát triển có SH; `--all` mọi run)
H1–H5 so với SH	`analysis/hsi_summary.py <dir> <oc dirs>` (báo cáo biến thể tên `HSI`; `paper/build_hsi_report.py` tự đổi tên)
tiêu chí đăng ký trước	`analysis/hsi_entry8_test.py <dir> HSI-VN HSI-VMN` (T1–T4 so với HSI-rot-m-sc)
quét α	`analysis/hsi_alpha.py <dir> --family HSI-VN HSI-VN --md --png`
sinh tập mới (bố cục lỗ mới)	job `hsi_fresh.py <ds> <desc> <seed0> <n> --keep navcache_rXX --variants oracle` (runner thêm thư mục dump) — chạy `nav_demo --cache --cache-only` rồi `hsi_eval`, giữ cache trong `/kaggle/working/navcache_rXX`
kéo file từ kernel nhiều acc	`automation/fetch_outputs.py <state.json> <dest> '<slug regex>' --pattern 'navcache_.*\.npz' --flatten` (token của chủ kernel)
publish dataset công khai	`automation/publish_dataset.py <dir> <slug> "<title 6–50>" "<subtitle 20–80>" --owner caotienminh1`
báo cáo	`paper/build_hsi_report.py <main dir> <oc dirs> --cmp name=dir[@v1,v2] ... --confirm <dir>` (tên `*_ho` = tập con có SH)
hình	`analysis/hsi_lockfig.py`, `analysis/hsi_alpha.py --png` (matplotlib phải `pip install` trong phiên)

3. Tổ chức thử nghiệm

- **Đăng ký trước mọi quyết định:** viết tiêu chí + quy tắc quyết định vào `RESEARCH_LOG.md`, commit + push TRƯỚC khi kernel kết quả xong; giờ ghi theo `date -u` của hệ thống (đã có lần ghi lệch 10 phút — đã sửa).
- **Phân chia dữ liệu:** thiết kế (seed 0 × 4 descriptor, phân tích cục bộ tuỳ ý) → phát triển (seed 0–9, chẩn đoán được) → **tập kiểm tra mới cho mỗi quyết định**. Đã dùng: seed 10–12, 13–15, 16–18. **Kho dự trữ seed 19–27** (`caotienminh1/hmm-navcache-r19`): rút một nhóm 3 seed chưa dùng cho mỗi quyết định, đánh dấu "đã dùng" trong `03_HANDOFF.md`. Lý do không dùng lại một tập: quá khớp thích nghi đo được (lợi ích +0.014 trên tập giữ ra dùng 6 lần co còn +0.007 trên tập mới).
- **Sinh cache một lần, publish, mount nhiều lần:** sinh 3 seed × 36 run mất ~1 giờ trên 18–25 phiên CPU (Boreas ~42 phút / seed là đường găng ⇒ mỗi seed Boreas một job); đánh giá một tập đã publish chỉ ~15 phút (6 kernel). Kernel sinh cache là private ⇒ kernel acc khác không mount được ⇒ phải kéo về + publish dataset công khai (không token, không dữ liệu nhạy cảm).
- **Đo trước khi chạy lớn:** mọi ý tưởng mới được đo trên tập thiết kế / vài run phát triển cục bộ (vài phút) trước khi dựng đợt Kaggle; 6 ý tưởng bị loại ở bước này (xem `01` §4).
- **Một cờ cho mỗi thay đổi** trong `online_v2` (mặc định = hành vi cũ) và một biến thể có tên trong `VARIANTS` ⇒ ablation và tái lập dễ; kiểm hồi quy: biến thể cũ cho đúng số cũ sau mỗi refactor.

4. Lỗi đã gặp và cách sửa (ý tưởng 2)

Lỗi	Triệu chứng	Sửa
runner `--nodumps`	FileNotFoundError khi in thư mục dump, mọi kernel hỏng trong 1 phút	`os.makedirs(dump_all, exist_ok=True)`
nhiều dataset cache cùng tên file	đợt Entry 4 mount 4 dataset trùng tên → đọc nhầm cache lỗ	token tường minh `DIR:navcache_@<th■ m■c>`; JSON ghi `cache_path`; xoá + dựng lại đợt
gom trộn kết quả	holes và main trùng khoá → ghi đè biến thể	`hsi_collect.py --tagre`
OpenMP quá tải	2 tiến trình GBM chậm ~10×	`OMP_NUM_THREADS`
băm đơn vị hiệu chuẩn	`(unit*k) % 2` cho mọi đơn vị CMU vào hiệu chuẩn	`calib_units` ngẫu nhiên tất định
cột toàn NaN	HistGradientBoosting "window shape cannot be larger"	`Model` bỏ cột toàn NaN
bảng báo cáo rỗng	biến thể chưa tồn tại trên tập cũ → 0 run	danh sách biến thể riêng cho từng bảng (`--cmp name=dir@v1,v2`)
tải nhầm cả dataset	`kaggle datasets download -f ""` (biến rỗng) kéo `archive.zip` 1.3 GB	luôn kiểm tên file trước; liệt kê bằng `kaggle datasets files`
lệnh `mkdir` sai thư mục	tạo thư mục rỗng trong repo, `rm -rf` xoá nhầm file vừa tải	dùng đường dẫn tuyệt đối, không chuỗi `cd` + lệnh xoá
`kaggle datasets create` trả 499	dataset vẫn tạo xong	kiểm bằng `datasets status` + đếm `datasets files`
worker phiên chat khởi động lại	monitor chết, kernel vẫn chạy trên Kaggle	khởi động lại monitor với `--pull-first "<regex>"` để kéo đầu ra đã xong
heredoc `'EOF'` + `\\n`	nhãn hình hiện "\n"	trong heredoc có nháy, viết `\n` một lần
background `until` + timeout 600 s	lệnh chờ bị chuyển nền rồi dừng	dùng `run_in_background` với `timeout` dài, hoặc Monitor `tail -F | grep`

5. Bài học thành công / thất bại

- **Thành công:** (1) đo cơ chế trên dữ liệu thiết kế trước (chẩn đoán vách khoá, hubness, đồng dạng) tiết kiệm hàng giờ Kaggle; (2) cache công khai + CPU ⇒ một đợt 441 run × 4–10 biến thể chỉ 15–40 phút, không tốn GPU; (3) đăng ký trước + tập mới ⇒ kết luận đứng vững (HSI-VN đạt 4/4 trên seed 16–18); (4) một monitor + `--pull-pattern` ⇒ không bị chặn API suốt phiên, không kéo file lớn.
- **Thất bại / cần tránh:** (1) tập thiết kế 28 run không lộ cái giá điều hướng của Entry 6 ⇒ luôn xem sai số điều hướng và đuôi p95, không chỉ recall; (2) tiêu chí cơ chế quá chặt (T3 Entry 8) làm một luật trội Pareto chưa được nhận — đặt tiêu chí theo thước đo kết quả, tiêu chí cơ chế để báo cáo; (3) giả định "lỗi điều hướng do chụm sai ngắn" sai — đo trước khi viết code.
- **Bảo mật:** token Kaggle chỉ ở `~/.kaggle/tok_<acc>` (chmod 600), không bao giờ trong repo / dataset / báo cáo / prompt; repo là public.